

Project Proposal

Mobile Servicing Shop ERP System

Full-Stack Developer — First Assessment Task

Prepared as an original technical design, based on independent research and engineering judgment.

1. Executive Summary

This proposal presents a complete, original design for an Enterprise Resource Planning (ERP) system for a mobile phone/device servicing shop, covering both frontend and backend scope as required for the Full-Stack Developer role.

The core insight driving this design is that a servicing shop's ERP is not a generic inventory-and-invoicing tool — its center of gravity is the Service Order: a single record that moves through a well-defined lifecycle (intake → diagnosis → approval → repair → quality control → billing → delivery → warranty), touching customers, technicians, parts stock, and money at each stage. Every module in this system exists to support, record, or enforce a transition in that lifecycle.

The recommended architecture is a modular monolith (not microservices): a single deployable backend organized into clear business modules (Identity, Customer/Device, Service Order, Inventory, Billing, Reporting), backed by one PostgreSQL database. This is a deliberate choice, justified in Section 12, based on the tightly coupled, transactional nature of the workflow — most operations here (e.g. approving a repair, consuming a part, closing an order) must be atomic across what would otherwise be several 'services' in a microservices split.

The frontend is a role-aware single-page application built around the Service Order as the primary object technicians, receptionists, and managers work with daily, supported by dedicated modules for customers, devices, inventory, billing, and reporting.

Sections 20 and 21 provide a phase-by-phase time estimate and a transparent, hours-based cost calculation in BDT, built from the exact scope documented here — not a generic ERP estimate.

2. Business & Domain Understanding

A mobile servicing shop repairs and maintains mobile phones and similar small electronics for walk-in and repeat customers. Unlike a generic retail ERP, its operational core is service delivery, not stock turnover — parts inventory exists to support repairs, not to be sold directly in most cases. The business has three concurrent concerns that the ERP must reconcile:

- Operational: tracking each device from intake to delivery without losing status visibility, across possibly several technicians and days.
- Financial: ensuring nothing is repaired without an approved estimate, no part leaves stock unrecorded, and every completed job is billed and reconciled.
- Trust: customers need transparency (what's wrong, what it costs, when it's ready) and the shop needs an audit trail to resolve disputes (e.g. "my phone had a working camera before you touched it").

This shapes several concrete design decisions used throughout this proposal: the Service Order as the central aggregate, mandatory approval before repair work, and inventory movements always being tied to a business reason (a service order or a manual adjustment) rather than a bare stock count.

3. Research Findings — What a Comprehensive Servicing-Shop ERP Typically Includes

Independent research into repair-shop and field-service management systems (general repair-shop POS/CRM tools, mobile-device-repair specific software, and general field-service/ERP literature) shows a recurring set of capability areas, which this proposal validates against and adapts rather than copies:

- Customer & device intake with searchable service history per device (by IMEI/serial), so repeat issues are visible.
- A structured repair pipeline with distinct diagnosis, estimate/approval, and repair stages — almost universally present because it protects the shop from doing unpaid or disputed work.
- Technician assignment and workload tracking, often with simple queues rather than complex scheduling engines for shops of this size.
- Parts/inventory tracking tied to jobs, with low-stock alerts, since stock-outs directly block repairs.
- Quality control as a distinct, auditable step before a device is marked ready — a pattern borrowed from general repair/QA workflows.
- Invoicing and payment tracking, frequently supporting partial/deposit payments.
- Warranty tracking on repairs (not just products), to handle repeat-issue claims.
- Role-based access (owner/manager vs front-desk vs technician) since staff turnover is common in this industry and access needs to be revocable per person.
- Reporting on turnaround time, technician productivity, and revenue by repair type — the numbers a shop owner actually checks weekly.

This research confirms the two diagrams already produced (End-to-End Workflow and Service Order State Machine) align with how the industry structures this problem, so this proposal builds directly on them rather than replacing them. Where this proposal diverges from generic templates is in the specifics: the exact state machine, the exact entity boundaries, and the decision to use a modular monolith rather than a heavier architecture — decisions made for this shop's scale, not copied from any single existing product.

4. Scope & Objectives

As a Full-Stack Developer submission, this proposal covers the complete system: frontend and backend. The objective is a production-oriented design — not a prototype, but a design intended to be directly implementable in production — for a single-branch mobile servicing shop, extensible later to multiple branches without a rewrite.

In scope

- Customer & device management, full service order lifecycle, technician workflow, inventory & parts consumption, billing & payments, warranty tracking, reporting, and user/role administration.

Out of scope (explicitly, to keep the proposal realistic and non-inflated)

- Multi-branch/franchise management, accounting-suite features (full general ledger, tax filing), e-commerce/online parts marketplace, and native mobile apps (a responsive web app covers technician/counter use — see Section 6 for the reasoning).

5. User Roles

Five actors are justified by the workflow; a Customer account was deliberately excluded from the ERP itself (see rationale below).

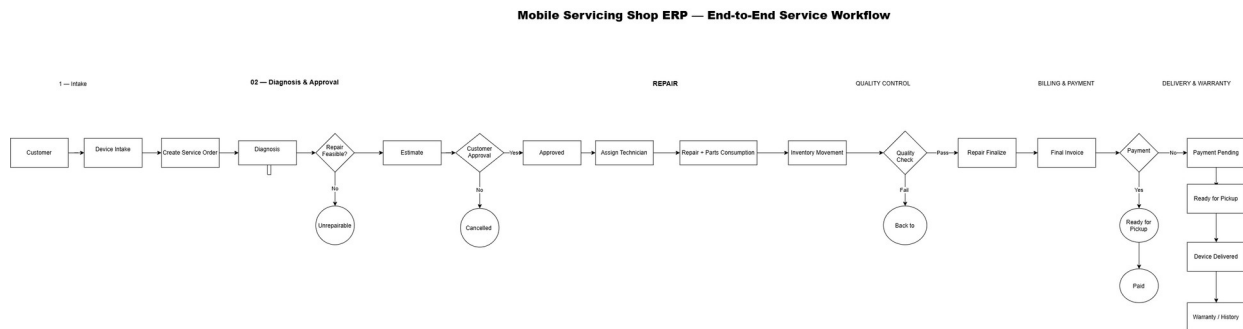
Role	Primary Responsibility	Key Permissions
Admin	System configuration & full oversight	All modules; user & role management; audit log access
Manager	Day-to-day shop operations	Approve high-value estimates, view reports, manage inventory levels, override QC in exceptional cases

Receptionist / Front Desk	Customer-facing intake & billing	Create customers/devices/service orders, record customer approval, generate invoices, take payments
Technician	Diagnosis & repair execution	Update assigned service orders, log diagnosis, consume parts, submit for QC
Customer	External party, not an ERP user	Receives approval requests & status updates via SMS/link; no login required (see below)

Rationale for not giving Customers an ERP account: at this shop's scale, customers interact with the business briefly and infrequently. Forcing account creation adds friction to intake and a security/privacy surface (customer credential management) with little benefit. Instead, customers approve estimates and check status via a signed, expiring link sent by SMS — no login, no password reset flows, same transparency. This can be upgraded to a full customer portal later if the business scales toward high repeat-customer volume, without changing the core data model.

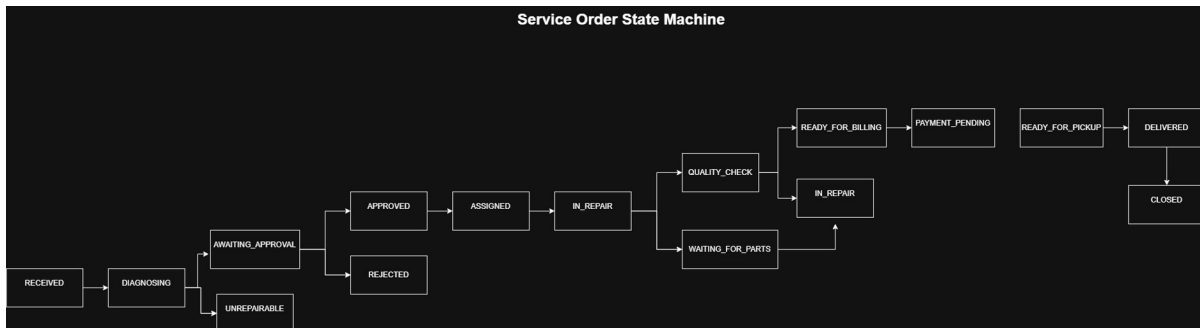
6. End-to-End Workflow

The following workflow was already established for this project and is reused as-is; independent review during this proposal confirmed it is complete and consistent with the state machine in Section 7 — no corrections were needed.



7. Service Order State Machine

This is the authoritative backend state model for a Service Order, reused unchanged from the existing work. Every business rule and API operation in this proposal is defined in terms of these states and their transitions.



From State	Event / Trigger	To State	Guard
RECEIVED	Technician starts diagnosis	DIAGNOSING	Device intake fields complete
DIAGNOSING	Diagnosis complete, feasible	AWAITING_APPROVAL	Estimate created
DIAGNOSING	Diagnosis complete, not	UNREPAIRABLE	—

	feasible		
AWAITING_APPROVAL	Customer approves	APPROVED	Valid, non-expired estimate
AWAITING_APPROVAL	Customer rejects / no response by deadline	REJECTED	—
APPROVED	Manager/receptionist assigns technician	ASSIGNED	Technician has capacity
ASSIGNED	Technician begins work	IN_REPAIR	—
IN_REPAIR	Required part unavailable	WAITING_FOR_PARTS	Available quantity_on_hand < quantity required by the RepairTask
WAITING_FOR_PARTS	Part received/restocked	IN_REPAIR	Available quantity_on_hand ≥ required quantity
IN_REPAIR	Technician finds additional required work	AWAITING_APPROVAL	New estimate version created for the additional work (BR-8)
AWAITING_APPROVAL	Customer approves the revised estimate	IN_REPAIR	Same technician resumes; no re-assignment needed
IN_REPAIR	Repair work finished	QUALITY_CHECK	All repair tasks marked done
QUALITY_CHECK	QC passes	READY_FOR_BILLING	Checklist fully passed
QUALITY_CHECK	QC fails	IN_REPAIR	At least one checklist item failed
READY_FOR_BILLING	Invoice generated	PAYMENT_PENDING	—
PAYMENT_PENDING	Payment recorded in full, or Manager override	READY_FOR_PICKUP	Amount paid ≥ invoice total, OR a Manager-authorized override with a mandatory logged reason (BR-5)
READY_FOR_PICKUP	Device handed to customer	DELIVERED	Signature/confirmation captured
DELIVERED	Order administratively closed	CLOSED	Independent of warranty period — Warranty has its own ACTIVE/EXPIRED lifecycle (Section 8)

Note on the two AWAITING_APPROVAL entry points: the state itself is reused rather than introducing a duplicate state, since the guard (a pending, unapproved estimate) is identical whether the order is being approved for the first time or re-approved mid-repair for additional work — see BR-8.

8. Key Modules & Features

Module	What it does
Customer & Device Management	Create/search customers, register devices per customer with IMEI/serial, view full device service history across multiple visits.
Service Order (core module)	Create/track orders through the full lifecycle, enforce the state machine, attach diagnosis notes, photos of device condition at intake (dispute protection).
Diagnosis & Estimation	Technician records findings and feasibility; system generates a versioned estimate (labor + parts) the customer must approve before repair starts.
Customer Approval	Signed, expiring link (SMS/WhatsApp) showing the estimate; approve/reject without requiring an account; full audit of decision + timestamp.
Technician Assignment & Workload	Simple queue view per technician; managers assign/reassign; technicians see only their assigned orders.
Repair Execution & Parts Consumption	Checklist of repair tasks; consuming a part deducts from inventory in the same transaction as marking the task done.
Inventory & Stock	Parts catalog, stock levels, reorder thresholds, low-stock alerts, full stock movement ledger (every unit in/out is traceable to a reason).
Quality Control	Mandatory checklist before an order can reach billing; failed QC sends the order back to repair with the failure reason attached.
Billing & Payments	Auto-generated invoice from the approved estimate + any adjustments; supports partial/deposit payments; payment must be settled before pickup.
Warranty & Service History	Warranty period per completed repair; a warranty claim reopens a linked new service order referencing the original, so history is never lost.

Reporting	Turnaround time per order, technician productivity, revenue by repair type/period, inventory valuation, low-stock report.
User & Role Administration	Manage staff accounts, assign roles, deactivate access immediately on staff departure.
Audit Log	Every state transition, price override, and stock adjustment is logged with actor, before/after, and timestamp — needed for dispute resolution and accountability.

9. Functional Requirements

ID	Requirement
FR-1	System shall allow front-desk staff to register a new customer and device, or attach a new service order to an existing device.
FR-2	System shall require a technician to record diagnosis findings and a feasibility flag before an estimate can be created.
FR-3	System shall permit repair tasks to be marked in progress only when the linked service order is in the ASSIGNED or IN_REPAIR state.
FR-4	System shall generate a signed, single-use, time-limited approval link for the customer (distinct from an authenticated user session) and record the approval decision with a timestamp.
FR-5	System shall reject any part-consumption request where the requested quantity exceeds the inventory item's current quantity_on_hand, checked and decremented within the same transaction.
FR-6	System shall require a passed Quality Check before an order can move to billing.
FR-7	System shall generate an invoice only from the latest approved estimate version plus documented, customer-approved adjustments — never from an arbitrary or unapproved amount.
FR-8	System shall not allow transitioning an order to READY_FOR_PICKUP while an outstanding invoice balance remains, unless a Manager records an authorized override with a mandatory reason.
FR-9	System shall track each Warranty as active until its valid_to date and expired thereafter; a claim filed while active shall create a new service order linked to the original order and device.
FR-10	System shall restrict all write actions by role-based permissions; Technicians shall only modify service orders assigned to them and shall have no access to billing or user-administration endpoints.
FR-11	System shall log every service-order state transition, inventory movement, manager override, and revised-estimate approval, with actor and timestamp.
FR-12	System shall provide search across customers, devices (by IMEI/serial), and service orders.
FR-13	System shall block continuation of repair work once additional required work materially increases the approved estimate, until the customer approves a revised estimate version.

10. Non-Functional Requirements

Category	Requirement
Security	Password hashing (bcrypt/argon2), JWT-based session tokens with short expiry + refresh, signed customer approval links with expiry, role-based authorization enforced server-side (never trust the client).
Data Integrity	Financial and stock-affecting operations (approve → invoice, consume part → stock deduction) run inside database transactions so partial failures cannot leave inconsistent state.
Reliability	Daily automated database backups; state-machine transitions validated server-side so an invalid transition is rejected regardless of what the UI sends.
Performance	Service order list and search return in well under 1 second at expected data volumes (a single shop generates hundreds, not millions, of orders per year); this does not need premature horizontal scaling.
Maintainability	Clear module boundaries inside the monolith (Section 12) so one module can be understood, tested, and modified without deep knowledge of the whole system.
Auditability	Every price override, manual stock adjustment, and state transition is attributable to a user and reversible in principle (nothing is hard-deleted; soft status changes only).
Scalability	Modular boundaries are designed so a high-load module (e.g. Reporting) could be split out later without redesigning the data model — see Section 24.

11. Business Rules

These rules are derived directly from the state machine (Section 7) and validated against the workflow — each has a concrete operational reason, not an arbitrary policy.

ID	Rule	Reason
BR-1	A repair cannot begin until the customer has approved the estimate.	Prevents unpaid/disputed work; matches AWAITING_APPROVAL → APPROVED gate.
BR-2	A part cannot be consumed if the requested quantity exceeds the inventory item's quantity_on_hand.	Protects inventory accuracy; enforced transactionally at the point of consumption, not just in the UI.
BR-3	An order cannot reach READY_FOR_BILLING without a passed Quality Check.	Matches QUALITY_CHECK → READY_FOR_BILLING transition; protects service quality.
BR-4	Failed QC returns the order to IN_REPAIR with the failure reason attached, not to any earlier state.	Avoids re-doing diagnosis/approval for a fixable QC issue.
BR-5	Full payment is required before an order becomes READY_FOR_PICKUP, unless a Manager records an authorized override with a mandatory reason.	Protects revenue while allowing real-world exceptions (e.g. trusted repeat customer) with accountability; the override is an exception to the transition guard, not a separate state.
BR-6	A rejected or unrepairable order is terminal for repair purposes but the device/customer record remains for history.	Keeps history intact even when a specific job doesn't proceed.
BR-7	Only the assigned technician (or a Manager/Admin) can update a service order's repair progress.	Prevents accidental or unauthorized cross-editing of another technician's job.
BR-8	If diagnosis during repair identifies additional work that materially increases the approved estimate, that work must not proceed until a revised estimate is created and re-approved by the customer.	Prevents unauthorized scope or price increases beyond what the customer originally agreed to; reuses the existing AWAITING_APPROVAL state and approval mechanism rather than inventing a new one.

12. System Architecture

Recommended architecture: a modular monolith, not microservices.

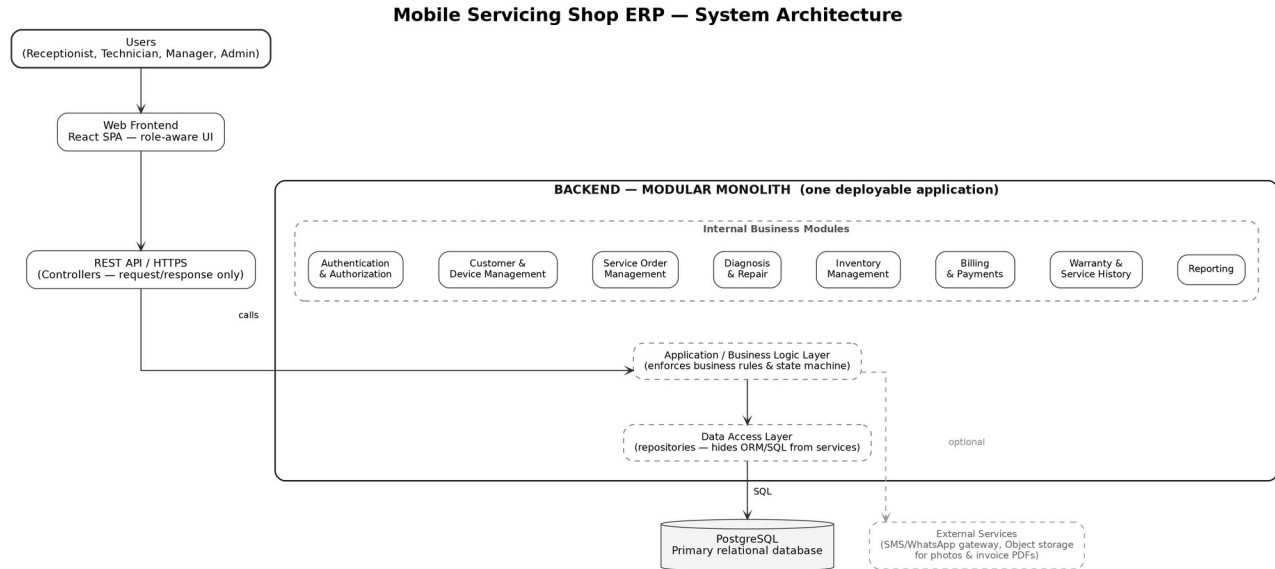
Why not microservices

Microservices are frequently the default assumption for anything called "ERP", but they solve problems this system does not have: independent scaling of unrelated services, independently deployed teams, and heterogeneous technology needs. This shop's core workflow is the opposite of independent — approving an estimate, consuming a part, and updating a service order's state are one business transaction that must succeed or fail together. Splitting them into separate services would force distributed transactions (sagas, eventual consistency) to simulate what a single database transaction gives for free, adding real operational complexity (service discovery, network failure handling, distributed tracing) for a system with modest data volume and a small team.

Why a modular monolith

- Transactional consistency: state transitions, stock deductions, and billing stay inside single DB transactions.
- Lower operational complexity: one deployable, one database to back up, one place to look for logs — appropriate for a small-to-mid engineering team.
- Clear module boundaries (Identity, Customer/Device, Service Order, Inventory, Billing, Reporting) still enforce separation of concerns in code, so the system does not become a tangled "big ball of mud".

- Future extraction is possible: if, for example, Reporting later needs heavy analytical load separate from transactional traffic, it can be extracted into its own service later — justified only if real scale demands it, not upfront.



A React SPA is chosen as the single client for both counter/reception use and technician use, accessed as a responsive web app on desktop and tablet. A native mobile app was considered but rejected for v1: the shop's technicians and staff work on-site with a fixed tablet/PC, not while mobile, so a responsive web app avoids the cost of maintaining a separate native codebase without a real usage benefit.

13. Backend Architecture & API Design

Layering

- Controllers — thin HTTP layer: parse request, call a service, shape the response. No business logic.
- Services / Use Cases — business logic lives here, one use case per meaningful business operation (see distinction below).
- Repositories — data access only, hide the ORM/SQL details from services.
- Validation — request-shape validation at the controller boundary (e.g. required fields, types); business-rule validation (Section 11) inside services.

CRUD vs. business operations

Simple reference data (customers, devices, inventory catalog entries) is exposed through conventional CRUD endpoints. The core workflow, however, is modeled as explicit business operations rather than generic "update service order" calls, because each one carries its own validation and side effects:

Representative Endpoint	Responsibility
POST /service-orders/{id}/approve	Transitions AWAITING_APPROVAL → APPROVED; validates the approval token/decision; rejects if estimate has expired.
POST /service-orders/{id}/consume-part	One transaction: verifies <code>quantity_on_hand</code> ≥ requested quantity, creates the <code>ServiceOrderPart</code> record, decrements <code>quantity_on_hand</code> , and writes the <code>StockMovement</code> audit row; rejects if insufficient stock (BR-2).
POST /service-orders/{id}/revised-estimate	Creates a new estimate version for additional work found mid-repair; transitions IN_REPAIR → AWAITING_APPROVAL; repair cannot resume until re-approved (BR-8).
POST /service-orders/{id}/quality-	Records checklist result; routes to READY_FOR_BILLING or back to IN_REPAIR

check	(BR-3/BR-4) — never a plain status field write.
POST /service-orders/{id}/invoice	Generates the Invoice from the latest approved Estimate version + logged adjustments; immutable once issued.
POST /invoices/{id}/payments	Records a payment; recalculates balance; triggers READY_FOR_PICKUP once balance reaches zero (BR-5).
POST /service-orders/{id}/pickup-override	Manager-only; transitions PAYMENT_PENDING → READY_FOR_PICKUP despite an outstanding balance; requires a mandatory reason, written to AuditLog (BR-5).

Authentication & Authorization

- JWT access tokens (short-lived) + refresh tokens; passwords hashed with bcrypt/argon2.
- Role-based access control enforced in middleware on every route, keyed to the roles in Section 5 — a Technician's token simply cannot call endpoints outside their assigned orders.
- The customer-facing approval link uses a separate, single-purpose signed token (not a login session) scoped to one estimate only.

Error handling & transactions

A consistent error envelope (code, message, field-level details for validation errors) is returned for all failures. Any operation that touches more than one table with a business invariant (approve → estimate lock, consume-part → stock deduction, payment → invoice balance) runs inside a single DB transaction so a failure midway leaves no partial state.

14. Database Architecture

A single relational database (PostgreSQL) is used — the workflow's strong consistency needs (Section 12) make a relational model the right fit; there is no case here for a NoSQL store other than possibly object storage for photos/PDF files.

Design refinement found during diagram review: the original stock ledger (StockMovement) carried a direct, nullable link to service_orders to represent "which order consumed this part." On review this conflated two different concerns — the append-only stock ledger, and the structured record of which parts (and at what price) belong to a given order's estimate/repair. A dedicated ServiceOrderPart table now sits between ServiceOrder and InventoryItem: it records quantity and unit_price per part per order (used to compute Estimate.parts_cost accurately before any stock is physically moved), while StockMovement links to ServiceOrderPart (instead of directly to ServiceOrder) purely to record the resulting stock quantity change. This is the one concrete correctness fix made during this revision; no other entities, relationships, or cardinalities changed.

Core tables & key considerations

Table	Key design decision
service_orders	Central table. FKs to customer, device, assigned technician. state column is a constrained enum, never freeform text — invalid states are a schema-level impossibility, not just an application check.
service_order_parts	New: join table between service_orders and inventory_items capturing quantity and unit_price at the time the part was estimated/used — this is what Estimate.parts_cost is actually computed from, independent of when/whether stock has physically moved yet.
inventory_items	quantity_on_hand is the single authoritative, transactionally-maintained stock count — not a value derived by summing stock_movements at query time. Every consumption decrements this column and writes a StockMovement row in the same transaction, so the column and the ledger can never drift apart.
stock_movements	Append-only audit ledger, never updated in place. Links to service_order_parts (nullable) when a movement is caused by a repair. This table records history only — it is not

	summed on the fly to determine current stock (see quantity_on_hand below).
estimates	Versioned: a new estimate row is created rather than editing an approved one in place, so the customer always approves an immutable snapshot.
invoices	Immutable once issued; adjustments after issuance are separate credit/adjustment records, not edits — needed for financial auditability.
audit_log	Generic (actor_id, entity, entity_id, action, before, after, at) table capturing all sensitive writes across modules via a shared service-layer hook, not duplicated logging code per module.

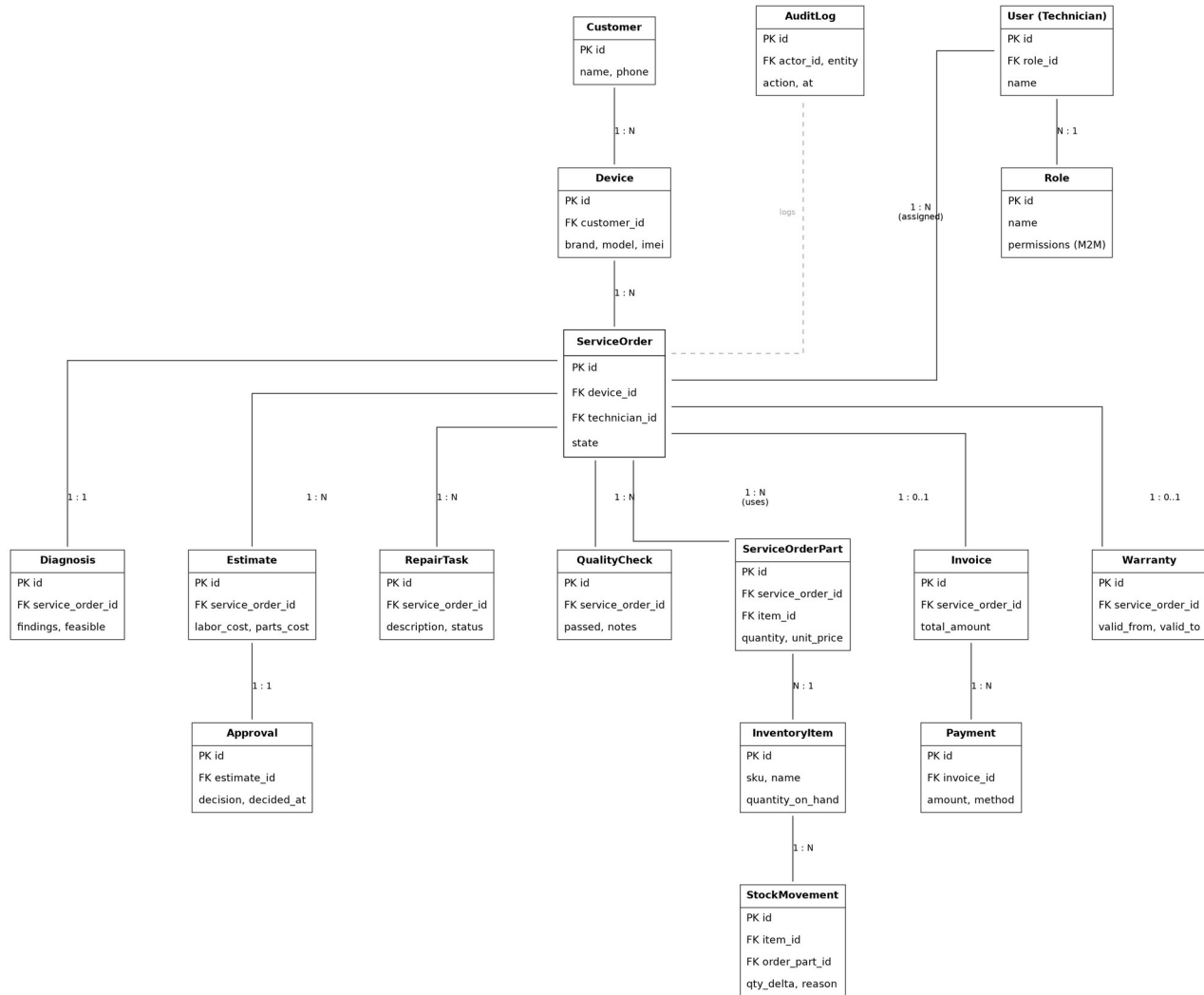
Constraints & indexes

- Foreign keys enforced at the DB level (not just application-level) between service_orders and customer/device/technician.
- Partial unique indexes on device.imei and device.serial_number, each applied only where the value is non-null — this prevents duplicate identity records for whichever identifier a given device actually has, without forcing every device to carry both.
- Composite index on service_orders(state, assigned_technician_id) to keep technician queue and dashboard queries fast.
- CHECK constraint quantity_on_hand >= 0 on inventory_items — enforced directly on the authoritative column itself, as a last line of defense beyond BR-2's application-level check. (A CHECK constraint cannot validate an aggregate SUM over an append-only ledger, which is exactly why quantity_on_hand is maintained as a real column rather than computed from stock_movements on the fly.)

Transactional boundaries

Each business operation from Section 13 (approve, consume-part, quality-check, invoice, payment, pickup-override) is wrapped in one database transaction covering every row it touches. Consuming a part, for example, is a single transaction that: (1) locks/checks the inventory row, (2) verifies quantity_on_hand $\geq$ the requested quantity, (3) writes the ServiceOrderPart record, (4) decrements quantity_on_hand, and (5) writes the StockMovement audit row — if any step fails, the entire operation rolls back, so the system never persists a state where stock was deducted but the consumption wasn't recorded, or vice versa.

Mobile Servicing Shop ERP — Entity Relationship Diagram



15. Frontend Architecture

Stack & organization

- React SPA, feature-based folder organization (each module — service-orders, customers, inventory, billing — owns its components, hooks, and API calls) rather than organizing by technical type (all components in one folder), so a developer can work within one module without touching unrelated code.
- Server state (data from the API: orders, customers, inventory) is managed with a dedicated data-fetching/caching layer (e.g. React Query) — this handles caching, refetching, and loading/error states consistently instead of hand-rolled fetch + useState in every screen.
- Local UI state (form inputs, modal open/closed, selected tab) stays in component-local React state — kept deliberately separate from server state so the two are never conflated.
- Forms use a schema-based validation library so client-side validation mirrors the same rules the backend enforces, giving fast feedback without duplicating rule logic by hand.

Role-aware UI

Navigation and available actions are derived from the logged-in user's role (Section 5) — a Technician's UI never renders a "Generate Invoice" button rather than rendering it and relying on the backend to reject the call; this is a UX decision layered on top of the mandatory server-side authorization, never a replacement for it.

Loading / error / empty states

Every data-driven screen defines three states explicitly: a skeleton/loading view, an error view with retry, and an empty view with a clear next action (e.g. an empty service-order queue shows "No orders assigned to you" rather than a blank table) — a small detail that matters heavily for daily-use operational software.

16. Information Architecture & Screens

Primary navigation (role-dependent visibility):

- Dashboard — today's queue, orders awaiting the current user's action, low-stock alerts (Manager/Admin).
- Service Orders — list (filterable by state/technician), detail view (the single most-used screen, showing the full lifecycle timeline for one order).
- Customers & Devices — search, profile with full device/service history.
- Inventory — stock list, stock movement history, low-stock report.
- Billing — invoices, payment recording.
- Reports — turnaround time, technician productivity, revenue.
- Admin — users, roles, permissions (Admin only).

Service Order detail screen (the core screen)

Structured as a vertical timeline mirroring the state machine (Section 7): each stage (Intake, Diagnosis, Estimate/Approval, Repair, QC, Billing, Delivery) is a section that is either completed (collapsed, showing a summary), active (expanded, showing the relevant action — e.g. the QC checklist), or not-yet-reached (greyed out). This directly reflects the backend state machine in the UI rather than presenting a flat form, so front-desk staff and technicians always know exactly what stage an order is in and what action is next.

Reusable components

- StateBadge — consistent color-coded rendering of a service order's state across every screen.
- DataTable — shared table component with sorting/filtering/pagination used by Service Orders, Customers, Inventory, and Reports lists.
- ApprovalStatusCard, StockLevelIndicator, MoneyInput (BDT-formatted) — small domain-specific components reused across screens rather than re-implemented per screen.

17. Service Order UX Walkthrough

Stage	UX Behavior
1. Device Intake	Receptionist searches for/creates the customer, registers the device (or selects an existing one), photographs its condition, and creates the service order — starts in RECEIVED.
2. Diagnosis	Technician opens their queue, records findings and a feasibility flag on the order's Diagnosis section — moves to DIAGNOSING then AWAITING_APPROVAL or UNREPAIRABLE.
3. Estimate	System (or technician, for non-standard repairs) builds the estimate from labor + required parts; parts availability is shown inline so infeasible estimates aren't sent.

4. Customer Approval	Customer receives an SMS link, sees the estimate and photos, taps Approve or Reject — no app or account needed.
5. Technician Assignment	Manager/receptionist assigns from the pool of available technicians shown with current workload — order moves to ASSIGNED.
6. Repair	Technician works through repair tasks; order is IN_REPAIR. If additional required work is found that materially changes the price, the technician raises a revised estimate directly from this screen; the order returns to "Awaiting your approval" (AWAITING_APPROVAL) and repair-task actions lock until the customer re-approves (BR-8).
7. Parts Consumption	Technician marks a task's part as consumed directly from the repair screen; quantity_on_hand deducts immediately and is visible to everyone (e.g. front desk sees when a part just ran out).
8. Quality Check	A separate reviewer (or the technician, for solo shops) runs the QC checklist; a failure keeps the order visibly in IN_REPAIR with the failure reason pinned to the top of the screen.
9. Final Billing	Invoice auto-generates from the latest approved estimate version plus any logged adjustments; receptionist reviews before issuing.
10. Payment	Payment recorded (supports partial); balance shown live. The "Ready for Pickup" action is disabled with a balance outstanding, unless a Manager uses the explicit override action, which requires typing a reason before it activates.
11. Pickup	Receptionist confirms handover; a simple signature/confirmation capture closes this step.
12. Delivery & Warranty	Order marked DELIVERED, then administratively CLOSED — independently of any warranty period. A Warranty record starts on delivery and is shown as active/expired based on its valid_to date; a later claim filed while active creates a new linked service order pre-filled with the original device and repair context.

18. Traceability: Business Need → Design

One worked example showing the chain maintained throughout this proposal (the same pattern applies to every rule in Section 11):

Layer	Content
Business Need	Don't repair a device the customer hasn't agreed to pay for.
Requirement	FR-4 — approval must be recorded via a signed token; FR-3 — repair tasks may only proceed from ASSIGNED/IN_REPAIR, both of which are unreachable without prior approval.
Business Rule	BR-1 — repair cannot begin until approved.
State Transition	AWAITING_APPROVAL → APPROVED (Section 7).
Domain Entities	Estimate, Approval, ServiceOrder.
API / Backend Behavior	POST /service-orders/{id}/approve — validates the single-purpose approval token (not an authenticated user session), locks the estimate version, transitions state (Section 13).
Frontend Behavior	Customer approval link screen (no login); ServiceOrder detail timeline shows "Awaiting your approval" until this call succeeds (Section 16/17).

19. Security, Authorization & Auditability Summary

Consolidated from Sections 10 and 13: role-based authorization enforced server-side on every route; hashed credentials; short-lived tokens; scoped single-purpose customer approval tokens; database-transaction-wrapped business operations; an append-only audit log covering every state transition, price override, and stock adjustment. Together these give the shop owner a defensible answer to "who changed this and when" for any record in the system.

20. Implementation Plan & Time Estimate

Verified against the final corrections in this revision (payment-override handling, the revised-estimate/re-approval sub-flow, and the quantity_on_hand integrity model): these are refinements to business rules and validation logic within the Service Order Core and Inventory/Billing phases already budgeted below, not new modules or screens. The estimate therefore stands unchanged from the prior draft.

Estimate is built bottom-up from the exact scope in this proposal (13 modules, ~12 core entities, role-aware frontend), for one full-stack developer working at a steady, sustainable pace with a realistic testing/documentation allowance — not a padded enterprise timeline and not an unrealistically compressed one.

Phase	Description	Hours
Discovery & Requirements Finalization	Confirm scope with stakeholder, finalize entities/rules (this document is the output of this phase).	16
UX/UI Design	Wireframes for all screens in Section 16, component inventory, review cycle.	32
Database & Backend Foundation	Schema migration, auth/RBAC, base repositories/services, module scaffolding.	40
Backend — Service Order Core	State machine engine, diagnosis/estimate/approval flow, business operation endpoints.	56
Backend — Inventory, Billing, Reporting	Stock ledger, invoicing/payments, reports, audit log wiring.	48
Frontend — Core Screens	Dashboard, Service Order list/detail timeline, Customer/Device screens.	56
Frontend — Inventory, Billing, Admin Screens	Remaining screens, role-aware navigation, reusable component library.	40
Integration & End-to-End Testing	Wiring frontend to real API, cross-module scenario testing (e.g. full order lifecycle), bug fixing.	40
Deployment & Handover Documentation	CI/CD setup, environment config, backup setup, handover docs.	16

Total estimated effort: 344 hours (≈ 43 working days at 8 hours/day, or roughly 9–10 calendar weeks for one full-time developer, allowing for normal review/feedback cycles).

21. Cost Estimate (BDT)

Calculation method: Estimated Hours × Hourly Rate = Phase Cost, summed across phases. The hourly rate reflects a mid-to-senior full-stack developer rate for BDT-market freelance/contract software delivery of production-oriented (not prototype) work, since the brief explicitly asks for the best solution without a budget constraint.

Assumed rate: BDT 900 / hour.

Phase	Hours	Cost (BDT)
Discovery & Requirements Finalization	16	BDT 14,400
UX/UI Design	32	BDT 28,800
Database & Backend Foundation	40	BDT 36,000
Backend — Service Order Core	56	BDT 50,400
Backend — Inventory, Billing, Reporting	48	BDT 43,200
Frontend — Core Screens	56	BDT 50,400
Frontend — Inventory, Billing, Admin Screens	40	BDT 36,000
Integration & End-to-End Testing	40	BDT 36,000
Deployment & Handover Documentation	16	BDT 14,400

Subtotal: BDT 309,600

Contingency (10%, for scope clarification / minor rework during review cycles): BDT 30,960

Total Project Cost: BDT 340,560

This figure covers full-stack development only, per the phases above; it excludes ongoing hosting/infrastructure costs and post-launch support, which would typically be quoted separately once real usage patterns are known.

22. Assumptions

- Single shop location for v1; multi-branch is a future enhancement (Section 24).
- Customers do not require ERP login accounts; SMS/link-based approval is sufficient (Section 5).
- PostgreSQL is an acceptable database choice (no existing infrastructure constraint communicated).
- SMS/notification delivery uses a standard third-party gateway (e.g. local SMS API) rather than being built in-house.
- One full-time developer for the estimate in Section 20; a team would compress calendar time but not proportionally reduce total effort due to coordination overhead.

23. Risks & Mitigations

Risk	Mitigation
Customer approval delays block the pipeline	Set an expiry on the approval link with automated follow-up reminders; allow staff to record an in-person/phone approval as a fallback channel.
Inventory data drifts from physical stock over time	Periodic manual stock-count reconciliation feature comparing physical count to the movement ledger, flagging discrepancies rather than silently trusting the system total.
Staff resistance to a new system replacing informal/paper processes	Design the Service Order detail screen to mirror the shop's existing mental model (a physical repair ticket) rather than introducing unfamiliar workflows.
Scope creep during development (common in fixed-price ERP projects)	The explicit out-of-scope list in Section 4 and the phase-based estimate in Section 20 make additions visible and separately quotable rather than silently absorbed.

24. Future Scalability & Enhancements

- Multi-branch support: add a branch_id scoping column across core tables — the modular structure means this is additive, not a rewrite.
- Customer portal: upgrade from link-based approval to full accounts once repeat-customer volume justifies the added complexity.
- Extracting Reporting into a separate read-optimized service if analytical query load starts affecting transactional performance — the module boundary in Section 12 already anticipates this.
- Native mobile app for technicians if the shop later adopts an on-the-go repair/pickup model (e.g. doorstep service).

25. Research Sources

The following sources informed general domain understanding during the research phase of this proposal (repair-ticket lifecycles, customer/device history, parts inventory, and RBAC concepts):

- RepairDesk — "Cellphone Repair Shop Software Features" (blog.repairdesk.co) — repair-ticket, inventory, and employee-management module patterns in repair-shop software.
- CellSmart POS — "Cell Phone Repair Shop Management Software Features" and "Mobile Repair Service Management Software" (cellsmartpos.com/blog) — repair ticketing, customer history, and inventory feature patterns.
- Orderry — "Cell Phone Repair Shop Software" (orderry.com/phone-repair-shop) — estimate/ticket workflow and status-sequence concepts.
- RO App — "Mobile Phone Repair Shop Software" (roapp.io) — parts inventory and stock-deduction patterns tied to completed repairs.
- NIST — Role Based Access Control glossary and NISTIR 6192 (csrc.nist.gov) — foundational RBAC concepts (roles, permissions, role-to-user assignment).

These sources were used for domain research only. The system architecture, requirements, state model, database design, and implementation approach in this proposal were independently derived from the assessment requirements and engineering judgment — no third-party feature list, architecture, or wording was copied or adapted as our own design.